

Documentation technique tres detaillee du module live NewsHub

Explication complete du fonctionnement du live, du moment exact ou un utilisateur rejoint le direct, du role de chaque fonction, et de l'usage combine de WebSocket et WebRTC.

Document genere le 08/05/2026 11:07. Cette documentation cible la partie live du projet present dans C:\Users\hp\Desktop\projet angular\interface_rapport\NewsHub.

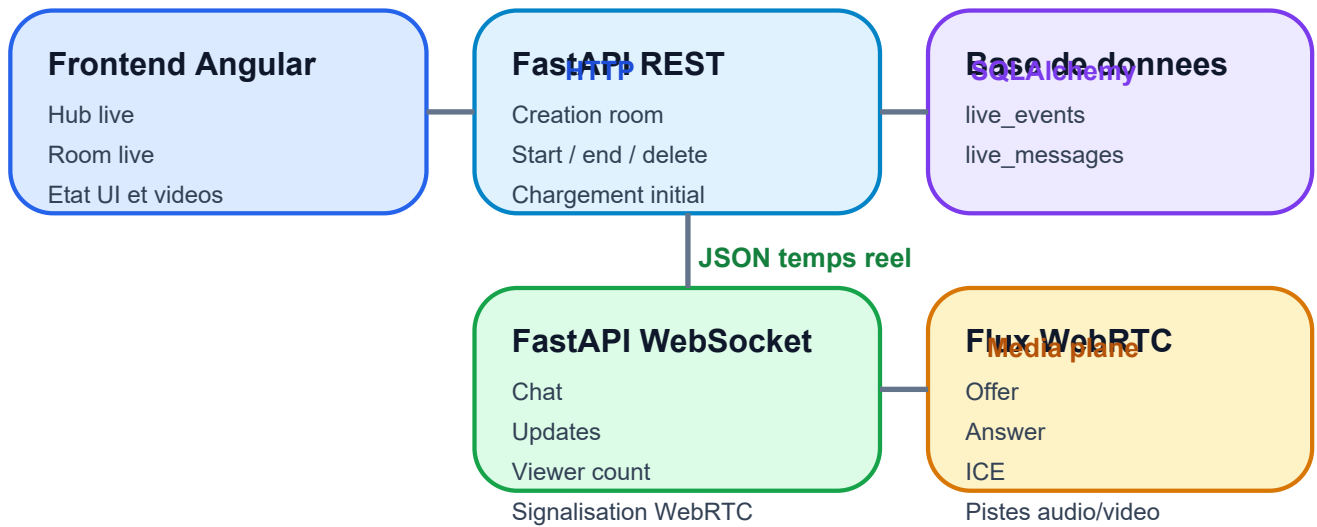
Ce PDF couvre: le parcours utilisateur depuis /live jusqu'a la reception du flux video, le contrat de messages temps reel, les endpoints REST, les modeles de donnees, les captures de code les plus importantes, et une lecture pas a pas de toutes les fonctions pertinentes de la fonctionnalite live.

Contenu principal	Valeur
Partie front	Angular: routes, services, hub live, live room, WebRTC cote navigateur
Partie back	FastAPI: endpoints REST, WebSocket, guard d'accès, manager de connexions
Temps reel	WebSocket pour le controle; WebRTC pour l'audio/video
Format de lecture	Explication architecturale + lecture ligne par ligne des blocs majeurs

1. Vue d'ensemble de l'architecture live

Le module live de NewsHub est volontairement separe en deux couches temps reel distinctes. La premiere couche est WebSocket: elle sert au controle de session, au chat, aux updates editoriales, au compteur de viewers et a la signalisation WebRTC. La seconde couche est WebRTC: elle sert exclusivement au transport du media, donc a la camera et au microphone de l'editeur vers les viewers.

Cette separation est importante. Le bouton Start live room ne transporte aucune video. Il bascule seulement la room en statut live via REST, puis le serveur pousse room_status aux clients connectes. La video ne démarre que quand l'editeur clique sur Go live with camera. A ce moment, la page met isBroadcasting a true, envoie broadcaster_ready via WebSocket, puis la sequence Offer / Answer / ICE du WebRTC peut commencer.



Schema simplifie de la separation entre couche de controle et couche media.

Fichiers pivots du module

- frontend/src/app/app.routes.ts ouvre les routes /live et /live/:id.
- frontend/src/app/core/models/live-event.model.ts fixe le contrat de donnees et le catalogue de messages socket.
- frontend/src/app/core/services/live-events.ts encapsule les appels REST du module.
- frontend/src/app/core/services/live-room-socket.ts ouvre le canal WebSocket et publie les evenements au composant.
- frontend/src/app/features/live/live-hub-page/live-hub-page.ts gere la creation et la liste des rooms.
- frontend/src/app/features/live/live-room-page/live-room-page.ts contient la logique metier centrale du lecteur, de l'editeur et du WebRTC.
- backend/models.py contient LiveEvent et LiveMessage.
- backend/main.py contient les endpoints REST live, le manager WebSocket et l'endpoint /ws/live-events/{event_id}.

API live exposee par le backend

Endpoint	Role
GET /live-events	Retourne toutes les live rooms, trieées avec les rooms live en tete et le viewer_count calcule a chaud.
GET /live-events/{event_id}	Charge le detail d'une room: metadonnees, updates recentes et chat recent.
POST /live-events	Creation d'une room par un utilisateur ayant le role editor.

Endpoint	Role
POST /live-events/{event_id}/start	Passe la room en statut live et notifie les clients via WebSocket.
POST /live-events/{event_id}/end	Passe la room en statut ended et emet stream_ended.
DELETE /live-events/{event_id}	Supprime la room et ses messages associes.
WS /ws/live-events/{event_id}?token=...	Canal temps reel pour chat, updates, viewer_count et signalisation WebRTC.

Messages temps reel du projet

Message	Emetteur	Recepteur	Utilite dans le projet
socket_ready	Serveur	Client qui vient de se connecter	Confirme l'ouverture de la socket, attribue clientId et renvoie le status courant de la room.
viewer_count	Serveur	Tous les clients de la room	Met a jour le compteur de connexions WebSocket actives.
room_status	Serveur	Tous les clients	Informe qu'une room passe a live ou ended.
chat_message	Client -> serveur -> room	Tous les clients	Diffuse un message de chat persiste en base.
live_update	Editeur -> serveur -> room	Tous les clients	Diffuse une update officielle de l'editeur, egalement persistee.
broadcaster_ready	Editeur	Lecteurs	Annonce que l'editeur a demarre la diffusion camera et que les lecteurs peuvent demander une connexion WebRTC.
viewer_joined	Lecteur	Editeur	Signale qu'un lecteur veut recevoir un flux video.
offer	Editeur	Lecteur cible	Premier SDP de negotiation WebRTC.
answer	Lecteur	Editeur cible	Reponse SDP a l'offre recue.
ice_candidate	Editeur ou lecteur	Pair cible	Echange des candidats ICE pour trouver un chemin reseau valide.
stream_ended	Editeur ou serveur	Tous les clients	Indique l'arret du flux camera et provoque le nettoyage cote front.

Ce qui est persistant et ce qui est seulement en memoire

Element	Stockage	Commentaire
LiveEvent	Base SQL	La room existe meme hors connexion WebSocket.
LiveMessage type=chat	Base SQL	Les messages de chat sont persists puis rebroadcastes.
LiveMessage type=update	Base SQL	Les updates officielles sont conservees comme historique editorial.
viewer_count	Memoire serveur	Valeur a chaud calculee a partir de live_room_manager.connections.
clientId socket	Memoire serveur	UUID genere a chaque connexion WebSocket.
RTCPeerConnection	Memoire navigateur	Objet purement client pour la negotiation et le transport media.
MediaStream local/remote	Memoire navigateur	Flux camera/micro de l'editeur et flux distant du lecteur.

2. Moment exact ou un utilisateur rejoint le live

La question la plus importante du projet live est la suivante: a quel moment le lecteur rejoint-il vraiment le direct ? Dans ce code, il existe en realite trois niveaux de jonction. Niveau 1: l'utilisateur ouvre la room sur la route /live/:id. Niveau 2: il rejoint logiquement la room quand la connexion WebSocket est acceptee et qu'il recoit le message socket_ready. Niveau 3: il rejoint visuellement le direct quand le flux WebRTC arrive dans le gestionnaire ontrack et que remoteVideo.srcObject recoit remoteStream.

Autrement dit, une room peut etre live sans que la video soit encore visible. C'est un detail tres important du projet: le statut de la room, le chat et les updates passent avant la connexion video. Le lecteur n'obtient le media que si l'editeur a effectivement active la camera et si la sequence viewer_joined -> offer -> answer -> ice_candidate -> ontrack s'est correctement terminee.



Le lecteur voit vraiment le direct a l'etape 7, quand le flux WebRTC arrive dans ontrack.

Les etapes 1 a 6 preparant l'accès, la socket et la signalisation, mais pas encore l'image.

Sequence logique complete qui mene le lecteur jusqu'au flux video.

Chronologie detaillee

- Le lecteur ouvre /live/:id. Angular lit l'id de route puis charge le detail de la room via getLiveEventById.
- Le composant evalue canJoinLiveRoom. S'il n'y a pas de session, showLoginGate s'affiche. Si la room est premium et que l'utilisateur n'est pas premium, showPremiumGate s'affiche.
- Quand canJoinLiveRoom devient vrai et qu'un JWT valide existe, attemptSocketConnection ouvre la connexion WebSocket.
- Le serveur valide le token, verifie que la room existe et controle l'accès premium via can_access_live_event.
- Apres accept(), le serveur renvoie socket_ready. A ce stade, le lecteur a rejoint la room logique et recupere clientId, viewerCount et le statut actuel.
- Si la room est deja en statut live, handleSocketEvent appelle requestViewerConnection. Sinon, le lecteur attend room_status ou broadcaster_ready.
- Quand l'editeur diffuse vraiment sa camera, il envoie broadcaster_ready. Les lecteurs redemandent alors une connexion en envoyant viewer_joined.
- L'editeur cree une offre SDP pour ce lecteur, le lecteur renvoie une answer, puis les deux cotes echangent les candidats ICE.
- Le moment precis ou le direct apparait est l'evenement ontrack dans createViewerPeerConnection. C'est la que remoteStream est affecte et que la video devient visible.

Difference essentielle entre Start live room et Go live with camera

Start live room modifie l'etat metier de la room en base et le serveur diffuse room_status = live. Go live with camera ne touche pas la base: il active la partie media cote front, pose isBroadcasting = true et envoie broadcaster_ready. Cette separation explique pourquoi un utilisateur peut voir 'The room is live' tout en attendant encore le message 'Waiting for

the editor camera stream'.

3. Pourquoi le projet utilise WebSocket et WebRTC en meme temps

Dans NewsHub, WebSocket est la couche de pilotage et WebRTC est la couche de transport audio/video. Cette architecture est saine: le serveur garde le controle de la room, du droit d'accès et de l'historique de messages, tandis que le navigateur optimise le transport media entre l'editeur et chaque lecteur.

WebSocket est bien adapte aux petits messages JSON de faible taille: chat_message, live_update, room_status, viewer_count, viewer_joined, offer, answer et ice_candidate. WebRTC, lui, est concu pour des pistes media temps reel, avec negotiation SDP, ICE et STUN. Le code montre bien cette repartition: aucune frame video n'est envoyee dans la WebSocket; seules les informations de controle et de signalisation y passent.

Technologie	Role dans NewsHub	Pourquoi elle est utile ici
REST HTTP	Creer, lister, demarrer, terminer et supprimer les rooms; charger l'etat initial	Adaptation naturelle aux actions metier et a la persistance en base.
WebSocket	Chat, updates, viewer_count, room_status et signalisation WebRTC	Canal bidirectionnel continu, leger, centralise par le serveur.
WebRTC	Transport du flux camera/micro de l'editeur	Faible latence, adaptation aux flux media, API native des navigateurs.

Ce que fait exactement WebSocket dans votre code

- Il authentifie l'entree dans la room grace au token passe dans l'URL ?token=....
- Il attribue un clientId unique par connexion.
- Il calcule le viewer_count et le diffuse a tous.
- Il persiste le chat et les updates via LiveMessage cote serveur.
- Il transporte les messages de signalisation WebRTC sans porter la video elle-meme.

Ce que fait exactement WebRTC dans votre code

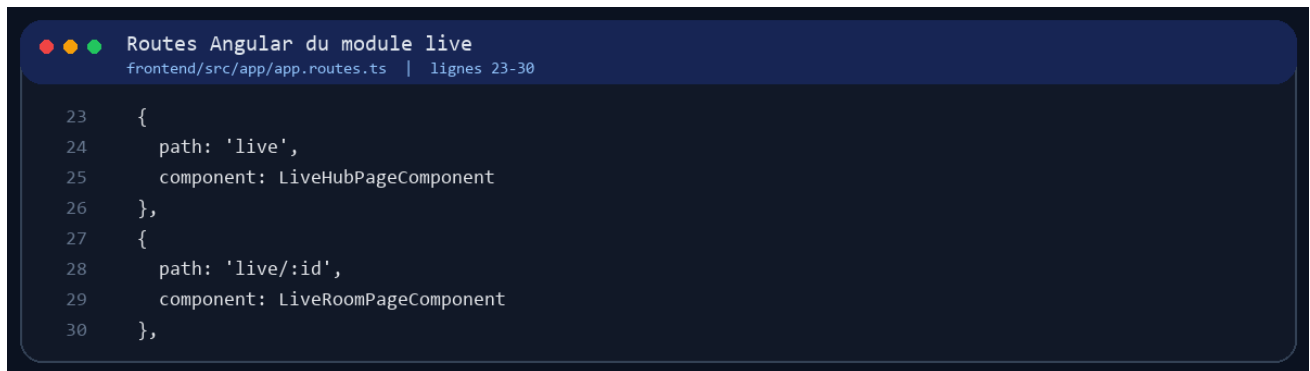
- Il cree une RTCPeerConnection cote editeur pour chaque lecteur actif dans broadcasterPeerConnections.
- Il cree une seule viewerPeerConnection cote lecteur pour recevoir le flux de l'editeur.
- Il utilise le serveur STUN public stun.l.google.com:19302 pour la decouverte reseau.
- Il ajoute les pistes de localStream avec addTrack cote editeur.
- Il declenche ontrack cote lecteur quand le flux distant devient disponible.

Observations techniques sur l'implementation actuelle

- Le champ stream_url existe dans LiveEvent, mais le flux actuel ne l'utilise pas: la diffusion reelle passe par la camera WebRTC du navigateur.
- Le serveur STUN est configure, mais aucun TURN n'est defini. Dans des reseaux tres contraints, certains lecteurs peuvent donc rencontrer des limites de connexion.
- Le compteur viewer_count compte les sockets actives, pas seulement les lecteurs ayant deja recu un flux video.
- Le service WebSocket Angular n'implemente pas de logique explicite onclose, onerror ou de reconnexion automatique.
- Le choix d'une connexion editeur -> une RTCPeerConnection par lecteur est simple et lisible, mais charge davantage la machine de l'editeur si l'audience augmente beaucoup.

4. Captures de code principales

Les captures ci-dessous servent d'ancrage visuel. Elles reprennent les blocs les plus importants du projet live et seront détaillées ensuite ligne par ligne.

A screenshot of a code editor window with a dark theme. The title bar at the top reads "Routes Angular du module live" and "frontend/src/app/app.routes.ts | lignes 23-30". The code is as follows:

```
23  {  
24    path: 'live',  
25    component: LiveHubPageComponent  
26  },  
27  {  
28    path: 'live/:id',  
29    component: LiveRoomPageComponent  
30  },
```

Routes Angular qui ouvrent le hub live et la room live.

```
43 export type LiveSocketEvent =  
44   | {  
45     type: 'socket_ready';  
46     clientId: string;  
47     viewerCount: number;  
48     status: LiveEventStatus;  
49     isEditor: boolean;  
50   }  
51   | {  
52     type: 'viewer_count';  
53     viewerCount: number;  
54   }  
55   | {  
56     type: 'room_status';  
57     status: LiveEventStatus;  
58   }  
59   | {  
60     type: 'chat_message';  
61     message: LiveMessage;  
62   }  
63   | {  
64     type: 'live_update';  
65     message: LiveMessage;  
66   }  
67   | {  
68     type: 'broadcaster_ready';  
69     senderClientId: string;  
70   }  
71   | {  
72     type: 'viewer_joined';  
73     senderClientId: string;  
74   }  
75   | {  
76     type: 'offer';  
77     senderClientId: string;  
78     sdp: RTCSessionDescriptionInit;  
79   }  
80   | {  
81     type: 'answer';  
82     senderClientId: string;  
83     sdp: RTCSessionDescriptionInit;  
84   }  
85   | {  
86     type: 'ice_candidate';  
87     senderClientId: string;  
88     candidate: RTCIceCandidateInit;  
89   }  
90   | {  
91     type: 'stream_ended';  
92   };
```

Union TypeScript qui decrit tous les evenements WebSocket du module.


```

Service Angular de connexion WebSocket
frontend/src/app/core/services/live-room-socket.ts | lignes 9-39

 9 export class LiveRoomSocketService {
10   private socket: WebSocket | null = null;
11   private readonly eventsSubject = new Subject<LiveSocketEvent>();
12
13   readonly events$: Observable<LiveSocketEvent> = this.eventsSubject.asObservable();
14
15   connect(roomId: number, token: string): void {
16     this.disconnect();
17
18     const wsBaseUrl = environment.backendApiBaseUrl.replace(/^http/i, 'ws');
19     const socketUrl = `${wsBaseUrl}/ws/live-events/${roomId}?token=${encodeURIComponent(token)}`;
20     this.socket = new WebSocket(socketUrl);
21
22     this.socket.onmessage = (event) => {
23       const message = JSON.parse(event.data) as LiveSocketEvent;
24       this.eventsSubject.next(message);
25     };
26   }
27
28   send(payload: object): void {
29     if (!this.socket || this.socket.readyState !== WebSocket.OPEN) {
30       return;
31     }
32
33     this.socket.send(JSON.stringify(payload));
34   }
35
36   disconnect(): void {
37     this.socket?.close();
38     this.socket = null;
39   }

```

Service Angular qui convertit l'URL HTTP en URL WebSocket et ouvre la connexion.

```

Creation d'une live room cote hub
frontend/src/app/features/live/live-hub-page/live-hub-page.ts | lignes 76-117

76  createLiveRoom(): void {
77    if (!this.isEditor) {
78      return;
79    }
80
81    this.createError = '';
82
83    if (this.createForm.invalid) {
84      this.createForm.markAllAsTouched();
85      return;
86    }
87
88    const formValue = this.createForm.getRawValue();
89    this.isCreating = true;
90
91    this.liveEventsService
92      .createLiveEvent({
93        title: formValue.title.trim(),
94        description: formValue.description.trim(),
95        category: formValue.category,
96        cover_image: formValue.coverImage.trim() || null,
97        premium_only: formValue.premiumOnly
98      })
99      .pipe(finalize(() => (this.isCreating = false)))
100      .subscribe({
101        next: (event) => {
102          this.createForm.reset(
103            {
104              title: '',
105              description: '',
106              category: 'technology',
107              coverImage: '',
108              premiumOnly: true
109            },
110            { emitEvent: false }
111          );
112          void this.router.navigate(['/live', event.id]);
113        },
114        error: (error) => {
115          this.createError = error.error?.detail || 'The live room could not be created right now.';
116        }
117      });

```

Bloc de creation d'une room depuis le hub live.

```

Initialisation et regles d'acces de la room
frontend/src/app/features/live/live-room-page/live-room-page.ts | lignes 80-156

80  ngOnInit(): void {
81    this.loginReturnUrl = this.router.url;
82    this.socketSubscription = this.liveRoomSocketService.events$.subscribe((event) => {
83      void this.handleSocketEvent(event);
84    });
85
86    this.authSubscription = this.authService.currentUser$.subscribe((user) => {
87      this.currentUser = user;
88      if (!user) {
89        this.disconnectSocket();
90        return;
91      }
92
93      this.attemptSocketConnection();
94    });
95
96    const eventId = Number(this.route.snapshot.paramMap.get('id'));
97    if (!eventId) {
98      this.error = 'Live room not found.';
99      this.loading = false;
100     return;
101   }
102
103   this.loadLiveRoom(eventId);
104 }
105
106 ngAfterViewInit(): void {
107   this.bindLocalStream();
108   this.bindRemoteStream();
109 }
110
111 ngOnDestroy(): void {
112   this.authSubscription?.unsubscribe();
113   this.socketSubscription?.unsubscribe();
114   this.stopCameraBroadcast({ keepRoomLive: true });
115   this.disconnectSocket();
116 }
117
118 get isPremiumUser(): boolean {
119   return !!this.currentUser?.isPremium;
120 }
121
122 get isEditor(): boolean {
123   return this.currentUser?.role === 'editor';
124 }
125
126 get isRoomEditor(): boolean {
127   return !!this.currentUser && !!this.event && this.currentUser.id === this.event.editor_user_id && this.isEditor;
128 }
129
130 get canJoinLiveRoom(): boolean {
131   if (!this.currentUser || !this.event) {
132     return false;
133   }
134
135   if (this.isRoomEditor) {
136     return true;
137   }
138
139   return this.event.premium_only ? this.isPremiumUser : true;
140 }
141
142 get showLoginGate(): boolean {
143   return !this.currentUser;
144 }
145
146 get showPremiumGate(): boolean {
147   return !!this.currentUser && !!this.event && !this.canJoinLiveRoom;
148 }
149
150 get canSendChat(): boolean {
151   return this.isSocketReady && this.canJoinLiveRoom;
152 }
153
154 get hasRemoteStream(): boolean {
155   return !!this.remoteStream;
156 }

```

Initialisation de la room et calcul des droits d'accès.

```

    Actions de l'éditeur: camera, start, end, updates, chat
    document.getElementById('live-room').addEventListener('click', () => {
        // Actions de l'éditeur: camera, start, end, updates, chat
        // 1. Préparation de la caméra
        async prepareCamera(): Promise<void> {
            if (!this.isRoomEditor) {
                return;
            }

            try {
                this.cameraError = '';
                if (!this.localStream) {
                    this.localStream = await navigator.mediaDevices.getUserMedia({
                        video: true,
                        audio: true
                    });
                }

                this.isCameraPrepared = true;
                this.streamMessage = this.event?.status === 'live'
                    ? 'Camera is ready, go live when you are comfortable.'
                    : 'Camera preview is ready. Start the room when the event begins.';
                this.bindLocalStream();
            } catch (error) {
                this.cameraError = 'The webcam or microphone could not be opened.';
            }
        }

        // 2. Démarrage de la diffusion en direct
        startLiveRoom(): void {
            if (!this.event || !this.isRoomEditor) {
                return;
            }

            this.isLoadingAction = true;
            this.liveEventService
                .startLiveEvent(this.event.id)
                .pipe(finish(() => { this.isLoadingAction = false }));
            .subscribe({
                next: (event) => {
                    if (this.event) {
                        this.event = {
                            ...this.event,
                            ...event
                        };
                    }
                    this.streamMessage = 'The room is now live. Start broadcasting when ready.';
                },
                error: (error) => {
                    this.error = error.error.detail || 'The live room could not be started.';
                }
            });
        }

        // 3. Arrêt de la diffusion en direct
        async goLiveWithCamera(): Promise<void> {
            if (!this.event || !this.isRoomEditor) {
                return;
            }

            if (!this.isCameraPrepared) {
                await this.prepareCamera();
            }

            if (!this.localStream) {
                return;
            }

            this.isBroadcasting = true;
            this.streamMessage = 'Broadcasting live from your camera.';
            this.liveRoomSocketService.send({ type: 'broadcaster_ready' });
        }

        // 4. Fin de la diffusion en direct
        endLiveRoom(): void {
            if (!this.event || !this.isRoomEditor) {
                return;
            }

            this.isLoadingAction = true;
            this.stopCameraBroadcast({ keepRoomLive: false });
            this.liveEventService
                .endLiveEvent(this.event.id)
                .pipe(finish(() => { this.isLoadingAction = false }));
            .subscribe({
                next: (event) => {
                    if (this.event) {
                        this.event = {
                            ...this.event,
                            ...event
                        };
                    }
                    this.streamMessage = 'This live session has ended.';
                },
                error: (error) => {
                    this.error = error.error.detail || 'The live room could not be ended.';
                }
            });
        }

        // 5. Suppression de la diffusion en direct
        deleteLiveRoom(): void {
            if (!this.event || !this.isRoomEditor) {
                return;
            }

            const confirmed = window.confirm(
                `Delete "${this.event.title}"? This will remove the room and all its live messages.`
            );
            if (!confirmed) {
                return;
            }

            this.isLoadingAction = true;
            this.stopCameraBroadcast({ keepRoomLive: false });
            this.disconnectSocket();
            this.liveEventService
                .deleteLiveEvent(this.event.id)
                .pipe(finish(() => { this.isLoadingAction = false }));
            .subscribe({
                next: () => {
                    window.location.href = '/live';
                },
                error: (error) => {
                    this.error = error.error.detail || 'The live room could not be deleted.';
                }
            });
        }

        // 6. Mise à jour du contenu de la diffusion en direct
        public editRoomUpdate(): void {
            const content = this.editorUpdateText.trim();
            if (!content || !this.isRoomEditor || !this.isSocketReady) {
                return;
            }

            this.liveRoomSocketService.send({
                type: 'live_update',
                content
            });
            this.editorUpdateText = '';
        }

        // 7. Envoi d'un message en direct
        sendChatMessage(): void {
            const content = this.chatText.trim();
            if (!content || !this.isSocketReady) {
                return;
            }

            this.liveRoomSocketService.send({
                type: 'chat_message',
                content
            });
            this.chatText = '';
        }
    });

```

Actions front de l'éditeur: preview camera, start, go live, end, updates et chat.

```

107 private loadLivestream(eventId: number): void {
108   this.loading = true;
109   this.error = '';
110
111   this.livestreamService
112     .getLivestreamById(eventId)
113     .pipe(finalize(() => (this.loading = false)))
114     .subscribe();
115     next(event) => {
116       this.event = event;
117       this.updates = [...event.updates].reverse();
118       this.chatMessages = [...event.chatMessages];
119       this.streamMessage =
120         event.status === 'live'
121         ? 'The room is live, wait for the editor video to connect.'
122         : event.status === 'ended'
123         ? 'This live session has ended.'
124         : 'The editor has not started the live session yet.';
125       this.attachSocketConnection();
126     };
127     error() => {
128       this.error = 'unable to load this live room right now.';
129     }
130   }
131 }
132
133 private attemptSocketConnection(): void {
134   if (this.event || this.currentUser || this.canJoinLiveRoom || this.isSocketReady || this.isSocketConnecting) {
135     return;
136   }
137
138   const token = this.authService.getToken();
139   if (!token) {
140     return;
141   }
142
143   this.connectionLabel = 'connecting...';
144   this.isSocketConnecting = true;
145   this.livestreamSocketService.connect(this.event.id, token);
146 }
147
148 private disconnectSocket(): void {
149   this.isSocketReady = false;
150   this.isSocketConnecting = false;
151   this.socketClientId = '';
152   this.connectionLabel = 'Offline';
153   this.livestreamSocketService.disconnect();
154 }
155
156 private async handleSocketEvent(event: LivestreamEvent): Promise<void> {
157   if (this.event) {
158     return;
159   }
160
161   if (event.type === 'socket_ready') {
162     this.socketClientId = event.clientId;
163     this.isSocketReady = true;
164     this.isSocketConnecting = false;
165     this.connectionLabel = 'connected';
166     this.event = {
167       ...this.event,
168       viewer_count: event.viewerCount,
169       status: event.status
170     };
171
172     if (this.isRoomEditor && this.event.status === 'live') {
173       this.requestViewerConnection();
174     }
175     return;
176   }
177
178   if (event.type === 'viewer_count') {
179     this.event = {
180       ...this.event,
181       viewer_count: event.viewerCount
182     };
183     return;
184   }
185
186   if (event.type === 'room_status') {
187     this.event = {
188       ...this.event,
189       status: event.status
190     };
191
192     if (event.status === 'live') {
193       this.streamMessage = this.livestreamEditor
194         ? 'The room is live. Start the camera broadcast whenever you are ready.'
195         : 'The room is live, waiting for the editor camera stream.';
196       this.requestViewerConnection();
197     } else if (event.status === 'ended') {
198       this.streamMessage = 'This live session has ended.';
199       this.clearRoomViewerState();
200     }
201     return;
202   }
203
204   if (event.type === 'chat_message') {
205     this.chatMessages = [...this.chatMessages, event.message];
206     return;
207   }
208
209   if (event.type === 'live_update') {
210     this.updates = [event.message, ...this.updates];
211     return;
212   }
213
214   if (event.type === 'broadcast_ready') {
215     if (this.livestreamEditor && this.event.status === 'live') {
216       this.broadcastToAllPeerConnections = false;
217       this.requestViewerConnection();
218     }
219     return;
220   }
221
222   if (event.type === 'viewer_joined' && this.isRoomEditor && this.isBroadcasting) {
223     await this.createOfferForViewer(event.senderClientId);
224     return;
225   }
226
227   if (event.type === 'offer' && this.isRoomEditor) {
228     await this.handleAnswerFor(event.senderClientId, event.offer);
229     return;
230   }
231
232   if (event.type === 'answer' && this.isRoomEditor) {
233     const peerConnection = this.broadcastToAllPeerConnections.get(event.senderClientId);
234     if (!peerConnection) {
235       return;
236     }
237     await peerConnection.setRemoteDescription(new RTCSessionDescription(event.offer));
238     return;
239   }
240
241   if (event.type === 'ice_candidate') {
242     if (this.isRoomEditor) {
243       const peerConnection = this.broadcastToAllPeerConnections.get(event.senderClientId);
244       if (!peerConnection) {
245         return;
246       }
247       await peerConnection.addIceCandidate(new RTCIceCandidate(event.candidate));
248       return;
249     }
250
251     if (this.viewerPeerConnection) {
252       await this.viewerPeerConnection.addIceCandidate(new RTCIceCandidate(event.candidate));
253     }
254     return;
255   }
256
257   if (event.type === 'stream_stopped') {
258     this.streamMessage = 'The editor has stopped the camera stream.';
259     this.clearRoomViewerState();
260   }
261 }

```

Traitement des evenements socket dans le composant de room.

```

Signalisation WebRTC cote Angular
Frontend/src/app/features/live/live-room-page/live-room-page.ts | lignes 465-563

465 private requestViewerConnection(): void {
466   if (this.isRoomEditor || !this.isSocketReady || this.hasRequestedViewerConnection) {
467     return;
468   }
469
470   this.hasRequestedViewerConnection = true;
471   this.liveRoomSocketService.send({
472     type: 'viewer_joined'
473   });
474 }
475
476 private async createOfferForViewer(targetClientId: string): Promise<void> {
477   if (!this.localStream) {
478     return;
479   }
480
481   const peerConnection = this.createBroadcasterPeerConnection(targetClientId);
482   const offer = await peerConnection.createOffer();
483   await peerConnection.setLocalDescription(offer);
484
485   this.liveRoomSocketService.send({
486     type: 'offer',
487     targetClientId,
488     sdp: offer
489   });
490 }
491
492 private createBroadcasterPeerConnection(targetClientId: string): RTCPeerConnection {
493   const existing = this.broadcasterPeerConnections.get(targetClientId);
494   if (existing) {
495     return existing;
496   }
497
498   const peerConnection = new RTCPeerConnection({
499     iceServers: [{ urls: 'stun:stun.l.google.com:19302' }]
500   });
501
502   this.localStream?.getTracks().forEach((track) => {
503     peerConnection.addTrack(track, this.localStream!);
504   });
505
506   peerConnection.onicecandidate = (event) => {
507     if (!event.candidate) {
508       return;
509     }
510
511     this.liveRoomSocketService.send({
512       type: 'ice_candidate',
513       targetClientId,
514       candidate: event.candidate.toJSON()
515     });
516   };
517
518   this.broadcasterPeerConnections.set(targetClientId, peerConnection);
519   return peerConnection;
520 }
521
522 private async handleViewerOffer(senderClientId: string, sdp: RTCSessionDescriptionInit): Promise<void> {
523   const peerConnection = this.createViewerPeerConnection(senderClientId);
524   await peerConnection.setRemoteDescription(new RTCSessionDescription(sdp));
525
526   const answer = await peerConnection.createAnswer();
527   await peerConnection.setLocalDescription(answer);
528
529   this.liveRoomSocketService.send({
530     type: 'answer',
531     targetClientId: senderClientId,
532     sdp: answer
533   });
534 }
535
536 private createViewerPeerConnection(targetClientId: string): RTCPeerConnection {
537   if (this.viewerPeerConnection) {
538     return this.viewerPeerConnection;
539   }
540
541   this.viewerPeerConnection = new RTCPeerConnection({
542     iceServers: [{ urls: 'stun:stun.l.google.com:19302' }]
543   });
544
545   this.viewerPeerConnection.ontrack = (event) => {
546     this.remoteStream = event.streams[0] ?? null;
547     this.bindRemoteStream();
548     this.streamMessage = 'connected to the editor video stream.';
549   };
550
551   this.viewerPeerConnection.onicecandidate = (event) => {
552     if (!event.candidate) {
553       return;
554     }
555
556     this.liveRoomSocketService.send({
557       type: 'ice_candidate',
558       targetClientId,
559       candidate: event.candidate.toJSON()
560     });
561   };
562
563   return this.viewerPeerConnection;

```

Blocs Angular qui realisent la negotiation WebRTC.

```
96 class LiveEvent(Base):
97     __tablename__ = "live_events"
98
99     id = Column(Integer, primary_key=True, index=True)
100     title = Column(String(200), nullable=False)
101     description = Column(Text, nullable=False)
102     category = Column(String(50), nullable=False)
103     cover_image = Column(String(500), nullable=True)
104     stream_url = Column(String(500), nullable=True)
105     status = Column(String(20), nullable=False, default="upcoming")
106     premium_only = Column(Boolean, nullable=False, default=True)
107     editor_user_id = Column(Integer, ForeignKey("users.id", ondelete="SET NULL"), nullable=True)
108     created_at = Column(DateTime, server_default=func.now())
109     started_at = Column(DateTime, nullable=True)
110     ended_at = Column(DateTime, nullable=True)
111
112     editor = relationship("User", back_populates="live_events")
113     messages = relationship("LiveMessage", back_populates="live_event", cascade="all, delete-orphan")
114
115
116 class LiveMessage(Base):
117     __tablename__ = "live_messages"
118
119     id = Column(Integer, primary_key=True, index=True)
120     live_event_id = Column(Integer, ForeignKey("live_events.id", ondelete="CASCADE"), nullable=False)
121     user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
122     message_type = Column(String(20), nullable=False)
123     content = Column(Text, nullable=False)
124     created_at = Column(DateTime, server_default=func.now())
125
126     live_event = relationship("LiveEvent", back_populates="messages")
127     user = relationship("User", back_populates="live_messages")
```

Modeles SQLAlchemy qui representent room et messages live.

```
Gestionnaire serveur des connexions WebSocket
backend/main.py | lignes 243-302

243 class LiveRoomManager:
244     def __init__(self) -> None:
245         self.connections: dict[int, dict[str, dict]] = defaultdict(dict)
246
247     async def connect(self, room_id: int, websocket: WebSocket, user: models.User) -> str:
248         client_id = str(uuid4())
249         self.connections[room_id][client_id] = {
250             "socket": websocket,
251             "user_id": user.id,
252             "role": user.role or "user",
253         }
254         return client_id
255
256     def disconnect(self, room_id: int, client_id: str) -> None:
257         room_connections = self.connections.get(room_id)
258         if room_connections is None:
259             return
260
261         room_connections.pop(client_id, None)
262         if not room_connections:
263             self.connections.pop(room_id, None)
264
265     def get_viewer_count(self, room_id: int) -> int:
266         return len(self.connections.get(room_id, {}))
267
268     async def broadcast(self, room_id: int, payload: dict, exclude_client_id: Optional[str] = None) -> None:
269         room_connections = self.connections.get(room_id, {})
270         disconnected_clients: list[str] = []
271
272         for client_id, connection in room_connections.items():
273             if exclude_client_id and client_id == exclude_client_id:
274                 continue
275
276             try:
277                 await connection["socket"].send_json(payload)
278             except Exception:
279                 disconnected_clients.append(client_id)
280
281         for client_id in disconnected_clients:
282             self.disconnect(room_id, client_id)
283
284     async def send_to_client(self, room_id: int, client_id: str, payload: dict) -> None:
285         room_connections = self.connections.get(room_id, {})
286         target = room_connections.get(client_id)
287         if not target:
288             return
289
290         try:
291             await target["socket"].send_json(payload)
292         except Exception:
293             self.disconnect(room_id, client_id)
294
295     async def broadcast_viewer_count(self, room_id: int) -> None:
296         await self.broadcast(
297             room_id,
298             {
299                 "type": "viewer_count",
300                 "viewerCount": self.get_viewer_count(room_id),
301             },
302         )
```

Gestionnaire serveur des connexions temps reel.

[illegible]

Endpoints REST de creation, start, end et delete.

```
970 @app.websocket("/ws/live-events/{event_id}")
971 async def live_event_socket(
972     websocket: WebSocket,
973     event_id: int,
974     token: str = Query(default=""),
975 ):
976     db = database.SessionLocal()
977     client_id: Optional[str] = None
978     room_id = event_id
979
980     try:
981         if not token:
982             await websocket.close(code=4401)
983             return
984
985         current_user = get_current_user_from_token(token, db)
986         live_event = db.query(models.LiveEvent).filter(models.LiveEvent.id == event_id).first()
987
988         if not live_event:
989             await websocket.close(code=4404)
990             return
991
992         if not can_access_live_event(live_event, current_user):
993             await websocket.close(code=4403)
994             return
995
996         await websocket.accept()
997         client_id = await live_room_manager.connect(room_id, websocket, current_user)
998
999         await websocket.send_json(
1000             {
1001                 "type": "socket_ready",
1002                 "clientId": client_id,
1003                 "viewerCount": live_room_manager.get_viewer_count(room_id),
1004                 "status": live_event.status,
1005                 "isEditor": live_event.editor_user_id == current_user.id and (current_user.role or "user") == "editor",
1006             }
1007         )
1008         await live_room_manager.broadcast_viewer_count(room_id)
```

Validation serveur avant acceptation de la socket.

```

Branches chat, update et broadcaster_ready cote serveur
backend/main.py | lignes 1010-1085

1010     while True:
1011         payload = await websocket.receive_json()
1012         message_type = str(payload.get("type", "")).strip()
1013
1014         if message_type == "chat_message":
1015             content = str(payload.get("content", "")).strip()
1016             if not content:
1017                 continue
1018
1019             live_message = models.LiveMessage(
1020                 live_event_id=live_event.id,
1021                 user_id=current_user.id,
1022                 message_type="chat",
1023                 content=content,
1024             )
1025             db.add(live_message)
1026             db.commit()
1027             db.refresh(live_message)
1028             live_message.user = current_user
1029
1030             await live_room_manager.broadcast(
1031                 room_id,
1032                 {
1033                     "type": "chat_message",
1034                     "message": serialize_live_message(live_message),
1035                 },
1036             )
1037             continue
1038
1039         if message_type == "live_update":
1040             ensure_live_room_editor(live_event, current_user)
1041             content = str(payload.get("content", "")).strip()
1042             if not content:
1043                 continue
1044
1045             live_message = models.LiveMessage(
1046                 live_event_id=live_event.id,
1047                 user_id=current_user.id,
1048                 message_type="update",
1049                 content=content,
1050             )
1051             db.add(live_message)
1052             db.commit()
1053             db.refresh(live_message)
1054             live_message.user = current_user
1055
1056             await live_room_manager.broadcast(
1057                 room_id,
1058                 {
1059                     "type": "live_update",
1060                     "message": serialize_live_message(live_message),
1061                 },
1062             )
1063             continue
1064
1065         if message_type == "broadcaster_ready":
1066             ensure_live_room_editor(live_event, current_user)
1067             await live_room_manager.broadcast(
1068                 room_id,
1069                 {
1070                     "type": "broadcaster_ready",
1071                     "senderClientId": client_id,
1072                 },
1073                 exclude_client_id=client_id,
1074             )
1075             continue
1076
1077         if message_type == "stream_ended":
1078             ensure_live_room_editor(live_event, current_user)
1079             await live_room_manager.broadcast(
1080                 room_id,
1081                 {
1082                     "type": "stream_ended",
1083                 },
1084             )
1085             continue

```

Traitement serveur des messages chat, update et broadcaster_ready.



```
1087         if message_type in {"viewer_joined", "offer", "answer", "ice_candidate"}:
1088             outgoing_payload = {
1089                 "type": message_type,
1090                 "senderClientId": client_id,
1091             }
1092
1093             if message_type in {"offer", "answer"}:
1094                 outgoing_payload["sdp"] = payload.get("sdp")
1095
1096             if message_type == "ice_candidate":
1097                 outgoing_payload["candidate"] = payload.get("candidate")
1098
1099             target_client_id = payload.get("targetClientId")
1100             if target_client_id:
1101                 await live_room_manager.send_to_client(room_id, str(target_client_id), outgoing_payload)
1102             else:
1103                 await live_room_manager.broadcast(
1104                     room_id,
1105                     outgoing_payload,
1106                     exclude_client_id=client_id,
1107                 )
1108         except WebSocketDisconnect:
1109             pass
1110         finally:
1111             if client_id is not None:
1112                 live_room_manager.disconnect(room_id, client_id)
1113                 await live_room_manager.broadcast_viewer_count(room_id)
1114             db.close()
```

Relai serveur des messages WebRTC et nettoyage final.

5. Lecture detaillee du frontend

5.1 Routage live Angular

Fichier: frontend/src/app/app.routes.ts | Lignes: 23-30

Ce bloc branche le module live dans le routeur Angular. C'est le premier point d'entree du parcours utilisateur.

```
Routes Angular du module live
frontend/src/app/app.routes.ts | lignes 23-30

23  {
24    path: 'live',
25    component: LiveHubPageComponent
26  },
27  {
28    path: 'live/:id',
29    component: LiveRoomPageComponent
30  },
```

Capture de code: 5.1 Routage live Angular

Lignes	Explication
23-26	La route /live charge LiveHubPageComponent. C'est la page catalogue des rooms, avec eventuellement le formulaire de creation si l'utilisateur est editor.
27-30	La route /live/:id charge LiveRoomPageComponent. La room se base ensuite sur cet id pour charger les details et ouvrir la socket ciblee.

5.2 Structures LiveEvent et payload de creation

Fichier: frontend/src/app/core/models/live-event.model.ts | Lignes: 1-40

Le debut du fichier decrit les structures metier principales de la partie live cote frontend.

Lignes	Explication
1	LiveEventStatus limite les etats possibles a upcoming, live et ended. Cette union simplifie les controles d'interface et reduit les fautes de frappe.
3-10	LiveMessage decrit un message persiste par le backend. Le front ne distingue que deux types: chat et update.
12-27	LiveEvent porte la metadonnee de room. viewer_count est fourni par le serveur, tandis que stream_url est present mais n'alimente pas la diffusion WebRTC actuelle.
29-32	LiveEventDetail enrichit la room avec deux listes: updates et chat_messages. C'est la forme renvoyee par GET /live-events/{id}.
34-40	CreateLiveEventPayload est le contrat d'entree pour la creation d'une room depuis le hub.

5.3 Contrat exact des messages socket

Fichier: frontend/src/app/core/models/live-event.model.ts | Lignes: 43-92

Cette capture montre uniquement l'union TypeScript LiveSocketEvent. Le tableau ci-dessous explique exactement les lignes visibles dans l'image.

```
Contrat de messages WebSocket cote Angular
frontend/src/app/core/models/live-event.model.ts | lignes 43-92

43 export type LiveSocketEvent =
44   | {
45     type: 'socket_ready';
46     clientId: string;
47     viewerCount: number;
48     status: LiveEventStatus;
49     isEditor: boolean;
50   }
51   | {
52     type: 'viewer_count';
53     viewerCount: number;
54   }
55   | {
56     type: 'room_status';
57     status: LiveEventStatus;
58   }
59   | {
60     type: 'chat_message';
61     message: LiveMessage;
62   }
63   | {
64     type: 'live_update';
65     message: LiveMessage;
66   }
67   | {
68     type: 'broadcaster_ready';
69     senderClientId: string;
70   }
71   | {
72     type: 'viewer_joined';
73     senderClientId: string;
74   }
75   | {
76     type: 'offer';
77     senderClientId: string;
78     sdp: RTCSessionDescriptionInit;
79   }
80   | {
81     type: 'answer';
82     senderClientId: string;
83     sdp: RTCSessionDescriptionInit;
84   }
85   | {
86     type: 'ice_candidate';
87     senderClientId: string;
88     candidate: RTCIceCandidateInit;
89   }
90   | {
91     type: 'stream_ended';
92   };
```

Capture de code: 5.3 Contrat exact des messages socket

Lignes	Explication
43-50	Premiere variante: socket_ready. Le serveur renvoie clientId, viewerCount, status et isEditor juste apres l'ouverture de la WebSocket.
51-58	viewer_count et room_status servent a synchroniser l'etat global de la room sans recharger la page.
59-66	chat_message et live_update encapsulent un LiveMessage complet pour pouvoir mettre a jour l'interface immediatement.

Lignes	Explication
67-74	broadcaster_ready et viewer_joined pilotent l'amorçage du flux video. Le premier annonce que l'editeur diffuse; le second demande un flux.
75-84	offer et answer transportent les descriptions SDP necessaires a la negotiation WebRTC.
85-92	ice_candidate et stream_ended terminent respectivement la decouverte reseau et l'arret propre du flux.

5.4 Service REST live Angular

Fichier: frontend/src/app/core/services/live-events.ts | Lignes: 14-40

Ce service isole toutes les requetes HTTP du module live. Il garde le composant propre et concentre les URLs dans un seul endroit.

Lignes	Explication
15-16	Le service injecte HttpClient et construit apiUrl a partir de environment.backendApiBaseUrl + /live-events.
18-20	getLiveEvents() appelle GET /live-events pour afficher la liste du hub.
22-24	getLiveEventById(eventId) charge le detail d'une room ouverte par le routeur.
26-28	createLiveEvent(payload) poste la room au backend. Le JWT sera ajoute automatiquement par l'interceptor HTTP.
30-32	startLiveEvent(eventId) bascule la room a live. Cette action ne diffuse pas encore de video.
34-36	endLiveEvent(eventId) ferme metierement la room et declenche ensuite des notifications socket cote serveur.
38-40	deleteLiveEvent(eventId) supprime une room existante.

5.5 Service WebSocket Angular

Fichier: frontend/src/app/core/services/live-room-socket.ts | Lignes: 9-39

Ce service maintient le canal bidirectionnel temps reel et transforme les messages JSON en observable RxJS.

```
Service Angular de connexion WebSocket
frontend/src/app/core/services/live-room-socket.ts | lignes 9-39

9 export class LiveRoomSocketService {
10   private socket: WebSocket | null = null;
11   private readonly eventsSubject = new Subject<LiveSocketEvent>();
12
13   readonly events$: Observable<LiveSocketEvent> = this.eventsSubject.asObservable();
14
15   connect(roomId: number, token: string): void {
16     this.disconnect();
17
18     const wsBaseUrl = environment.backendApiBaseUrl.replace(/^http/i, 'ws');
19     const socketUrl = `${wsBaseUrl}/ws/live-events/${roomId}?token=${encodeURIComponent(token)}`;
20     this.socket = new WebSocket(socketUrl);
21
22     this.socket.onmessage = (event) => {
23       const message = JSON.parse(event.data) as LiveSocketEvent;
24       this.eventsSubject.next(message);
25     };
26   }
27
28   send(payload: object): void {
29     if (!this.socket || this.socket.readyState !== WebSocket.OPEN) {
30       return;
31     }
32
33     this.socket.send(JSON.stringify(payload));
34   }
35
36   disconnect(): void {
37     this.socket?.close();
38     this.socket = null;
39   }
}
```

Capture de code: 5.5 Service WebSocket Angular

Lignes	Explication
10-13	socket conserve l'instance WebSocket active, tandis que eventsSubject sert de pont vers le composant via events\$.
15-16	connect(roomId, token) commence par fermer proprement une ancienne socket pour eviter les doubles connexions.
18-20	Le code convertit l'URL backend HTTP en WS, ajoute l'id de room et le token encode dans la query string, puis ouvre la WebSocket.
22-25	Chaque message reçu est parse en JSON puis publie dans eventsSubject pour que LiveRoomPageComponent le traite.
28-34	send(payload) refuse d'envoyer si la socket n'est pas OPEN; cela evite des erreurs quand l'etat reseau n'est pas encore pret.
36-39	disconnect() ferme la socket puis remet la reference a null.

5.6 Service d'authentification utile au live

Fichier: frontend/src/app/core/services/auth/auth.ts | Lignes: 16-31, 38-49, 80-99

Le module live depend de deux comportements d'AuthService: la diffusion de currentUser\$ et la recuperation d'un JWT encore valide pour la WebSocket.

Lignes	Explication
16-31	Le constructeur recharge l'utilisateur et le token depuis localStorage. Si l'etat est incoherent ou si le token est expire, il nettoie le stockage et republie un utilisateur null.
38-49	getToken() est critique pour le live: sans JWT valide, attemptSocketConnection ne peut pas ouvrir /ws/live-events/{id}.
44-46	Si le token est expire, le service deconnecte l'utilisateur avant de renvoyer null.
80-99	isTokenExpired() decode le JWT cote navigateur, lit exp et compare a Date.now(). Cette verification se fait avant l'ouverture WebSocket.

5.7 Vue d'ensemble de LiveHubPageComponent

Fichier: frontend/src/app/features/live/live-hub-page/live-hub-page.ts | Lignes: 28-75 et 120-162

Le hub live est la porte d'entree du module. Cette partie couvre les etats, les permissions et le chargement de la liste, hors bloc exact de creation.

Lignes	Explication
29-32	Le composant injecte AuthService, LiveEventsService, FormBuilder et Router.
34-40	Ces etats pilotent la page: utilisateur courant, rooms chargees, loading, erreurs, creation en cours et suppression en cours.
42-58	categories et createForm definissent les options de creation. premiumOnly vaut true par default, ce qui montre que la fonctionnalite vise surtout le public premium.
60-65	ngOnInit() s'abonne a currentUser\$ puis lance loadLiveEvents(). Le hub ne depend pas du live en cours pour exister.
68-70	isEditor est un getter simple mais essentiel pour masquer ou afficher les outils de creation.
72-74	isOwnedByCurrentEditor(event) verifie qu'une room appartient a l'editeur courant, utile pour afficher le bouton Delete.
120-145	deleteLiveRoom(event) demande une confirmation, appelle le backend puis retire la room de liveEvents si la suppression reussit.
147-162	loadLiveEvents() charge le catalogue des rooms et gere les etats loading/error du hub.

5.8 Creation d'une room depuis le hub

Fichier: frontend/src/app/features/live/live-hub-page/live-hub-page.ts | Lignes: 76-117

Cette capture correspond uniquement a la fonction createLiveRoom(), donc le tableau ci-dessous explique exactement les lignes visibles dans l'image.

Creation d'une live room cote hub

frontend/src/app/features/live/live-hub-page/live-hub-page.ts | lignes 76-117

```
76  createLiveRoom(): void {
77    if (!this.isEditor) {
78      return;
79    }
80
81    this.createError = '';
82
83    if (this.createForm.invalid) {
84      this.createForm.markAllAsTouched();
85      return;
86    }
87
88    const formValue = this.createForm.getRawValue();
89    this.isCreating = true;
90
91    this.liveEventsService
92      .createLiveEvent({
93        title: formValue.title.trim(),
94        description: formValue.description.trim(),
95        category: formValue.category,
96        cover_image: formValue.coverImage.trim() || null,
97        premium_only: formValue.premiumOnly
98      })
99      .pipe(finalize(() => (this.isCreating = false)))
100      .subscribe({
101        next: (event) => {
102          this.createForm.reset(
103            {
104              title: '',
105              description: '',
106              category: 'technology',
107              coverImage: '',
108              premiumOnly: true
109            },
110            { emitEvent: false }
111          );
112          void this.router.navigate(['/live', event.id]);
113        },
114        error: (error) => {
115          this.createError = error.error?.detail || 'The live room could not be created right now.';
116        }
117      });
118  }
```

Capture de code: 5.8 Creation d'une room depuis le hub

Lignes	Explication
76-79	La creation est refusee si l'utilisateur courant n'est pas editor.
81-86	Le code vide createError puis valide le formulaire. En cas d'erreur, markAllAsTouched() force l'affichage des validations.
88-99	Le formulaire est lu, normalise avec trim() puis envoye a createLiveEvent().
100-117	En succes, le formulaire est reinitialise puis l'utilisateur est redirige vers /live/{event.id}. En erreur, createError est alimente avec le message du backend.

5.9 LiveRoomPageComponent: cycle de vie et controle d'accès

Fichier: frontend/src/app/features/live/live-room-page/live-room-page.ts | Lignes: 80-156

Ce bloc decide si l'utilisateur peut entrer, quand la socket doit s'ouvrir et quels panneaux d'interface doivent s'afficher.

```

Initialisation et regles d'acces de la room
frontend/src/app/features/live/live-room-page/live-room-page.ts | lignes 80-156

80  ngOnInit(): void {
81    this.loginReturnUrl = this.router.url;
82    this.socketSubscription = this.liveRoomSocketService.events$.subscribe((event) => {
83      void this.handleSocketEvent(event);
84    });
85
86    this.authSubscription = this.authService.currentUser$.subscribe((user) => {
87      this.currentUser = user;
88      if (!user) {
89        this.disconnectSocket();
90        return;
91      }
92      this.attemptSocketConnection();
93    });
94
95    const eventId = Number(this.route.snapshot.paramMap.get('id'));
96    if (!eventId) {
97      this.error = 'Live room not found.';
98      this.loading = false;
99      return;
100    }
101    this.loadLiveRoom(eventId);
102  }
103
104  ngAfterViewInit(): void {
105    this.bindLocalStream();
106    this.bindRemoteStream();
107  }
108
109  ngOnDestroy(): void {
110    this.authSubscription?.unsubscribe();
111    this.socketSubscription?.unsubscribe();
112    this.stopCameraBroadcast({ keepRoomLive: true });
113    this.disconnectSocket();
114  }
115
116  get isPremiumUser(): boolean {
117    return !!this.currentUser?.isPremium;
118  }
119
120  get isEditor(): boolean {
121    return this.currentUser?.role === 'editor';
122  }
123
124  get isRoomEditor(): boolean {
125    return !!this.currentUser && !!this.event && this.currentUser.id === this.event.editor_user_id && this.isEditor;
126  }
127
128  get canJoinLiveRoom(): boolean {
129    if (!this.currentUser || !this.event) {
130      return false;
131    }
132    if (this.isRoomEditor) {
133      return true;
134    }
135    return this.event.premium_only ? this.isPremiumUser : true;
136  }
137
138  get showLoginGate(): boolean {
139    return !this.currentUser;
140  }
141
142  get showPremiumGate(): boolean {
143    return !!this.currentUser && !!this.event && !this.canJoinLiveRoom;
144  }
145
146  get canSendChat(): boolean {
147    return this.isSocketReady && this.canJoinLiveRoom;
148  }
149
150  get hasRemoteStream(): boolean {
151    return !!this.remoteStream;
152  }
153
154  }
155
156

```

Capture de code: 5.9 LiveRoomPageComponent: cycle de vie et controle d'acces

Lignes	Explication
80-84	ngOnInit() memorise l'URL courante pour un retour apres login puis s'abonne au flux events\$ du service WebSocket.
86-94	Le composant s'abonne a currentUser\$. Si l'utilisateur disparaît, disconnectSocket() est appele. Sinon, on tente une connexion socket.
96-103	L'id de route est lu depuis ActivatedRoute. Sans id valide, la room s'arrete immediatement avec une erreur.
106-109	ngAfterViewInit() essaye de binder les flux video aux balises video des que la vue existe.

Lignes	Explication
111-116	ngOnDestroy() desabonne, coupe la diffusion locale si necessaire et ferme la socket.
118-128	isPremiumUser, isEditor et isRoomEditor definissent les roles de base. isRoomEditor exige a la fois le role editor et la propriete editor_user_id de la room.
130-140	canJoinLiveRoom() contient la vraie regle d'entree: il faut un currentUser et une room; l'editeur propriétaire entre toujours; sinon le premium est verifie si la room est premium_only.
142-148	showLoginGate et showPremiumGate pilotent l'affichage des portes d'entree UI.
150-155	canSendChat et hasRemoteStream sont des raccourcis de confort pour les boutons et overlays.

5.10 Actions de l'editeur et du chat dans la room

Fichier: frontend/src/app/features/live/live-room-page/live-room-page.ts | Lignes: 158-305

Ici se trouvent les actions que l'utilisateur voit directement: camera preview, start de la room, bascule camera live, end, delete, updates editoriales et chat.

```

158 async prepareCamera(): Promise<void> {
159   if (!this.isRoomEditor) {
160     return;
161   }
162   try {
163     this.cameraError = '';
164     if (!this.localStream) {
165       this.localStream = await navigator.mediaDevices.getUserMedia({
166         video: true,
167         audio: true
168       });
169     }
170     this.isCameraPrepared = true;
171     this.sendMessage = this.event?.status === 'live'
172       ? 'Camera is ready, be live when you are comfortable.'
173       : 'Camera preview is ready. Start the room when the event begins.';
174     this.unbindLocalStream();
175   } catch {
176     this.cameraError = 'The webcam or microphone could not be opened.';
177   }
178 }
179
180 startLiveRoom(): void {
181   if (!this.event || !this.isRoomEditor) {
182     return;
183   }
184   this.isLoadingAction = true;
185   this.liveEventService
186     .startLiveEvent(this.event.id)
187     .pipe(finish(() => { this.isLoadingAction = false })))
188     .subscribe({
189       next: (event) => {
190         if (this.event) {
191           this.event = {
192             ...this.event,
193             ...event
194           };
195           this.sendMessage = 'The room is now live. Start broadcasting when ready.';
196         },
197         error: (error) => {
198           this.error = error.error.detail || 'The live room could not be started.';
199         }
200       });
201   }
202 }
203
204 async goLiveWithCamera(): Promise<void> {
205   if (!this.event || !this.isRoomEditor) {
206     return;
207   }
208   if (!this.isCameraPrepared) {
209     await this.prepareCamera();
210   }
211   if (!this.localStream) {
212     return;
213   }
214   this.isBroadcasting = true;
215   this.sendMessage = 'Broadcasting live from your camera.';
216   this.liveRoomSocketService.send({ type: 'broadcaster_ready' });
217 }
218
219 endLiveRoom(): void {
220   if (!this.event || !this.isRoomEditor) {
221     return;
222   }
223   this.isLoadingAction = true;
224   this.stopCameraBroadcast({ keepRoomLive: false });
225   this.liveEventService
226     .endLiveEvent(this.event.id)
227     .pipe(finish(() => { this.isLoadingAction = false })))
228     .subscribe({
229       next: (event) => {
230         if (this.event) {
231           this.event = {
232             ...this.event,
233             ...event
234           };
235           this.sendMessage = 'This live session has ended.';
236         },
237         error: (error) => {
238           this.error = error.error.detail || 'The live room could not be ended.';
239         }
240       });
241   }
242 }
243
244 deleteLiveRoom(): void {
245   if (!this.event || !this.isRoomEditor) {
246     return;
247   }
248   const confirmed = window.confirm(
249     `Delete "${this.event.title}"? This will remove the room and all its live messages.`
250   );
251   if (!confirmed) {
252     return;
253   }
254   this.isLoadingAction = true;
255   this.stopCameraBroadcast({ keepRoomLive: false });
256   this.disconnectSocket();
257   this.liveEventService
258     .deleteLiveEvent(this.event.id)
259     .pipe(finish(() => { this.isLoadingAction = false })))
260     .subscribe({
261       next: () => {
262         window.location.href = '/live';
263       },
264       error: (error) => {
265         this.error = error.error.detail || 'The live room could not be deleted.';
266       }
267     });
268 }
269
270 publishEditorUpdate(): void {
271   const content = this.editorUpdateText.trim();
272   if (!content || !this.isRoomEditor || !this.isSocketReady) {
273     return;
274   }
275   this.liveRoomSocketService.send({
276     type: 'live_update',
277     content
278   });
279   this.editorUpdateText = '';
280 }
281
282 sendChatMessage(): void {
283   const content = this.chatText.trim();
284   if (!content || !this.isRoomChat) {
285     return;
286   }
287   this.liveRoomSocketService.send({
288     type: 'chat_message',
289     content
290   });
291   this.chatText = '';
292 }

```

Capture de code: 5.10 Actions de l'editeur et du chat dans la room

Lignes	Explication
158-180	prepareCamera() n'est autorise que pour l'editeur propriétaire. Elle ouvre getUserMedia(video+audio), prepare localStream, active isCameraPrepared puis met a jour le message d'etat.
182-205	startLiveRoom() appelle l'endpoint REST /start. Son but est uniquement de faire passer le statut de la room a live et de mettre a jour l'interface.
207-223	goLiveWithCamera() exige une room, un editeur et un localStream. Elle met isBroadcasting a true puis envoie broadcaster_ready. C'est l'etape qui lance la partie media.

Lignes	Explication
225-250	<code>endLiveRoom()</code> coupe d'abord la diffusion locale avec <code>stopCameraBroadcast()</code> , puis appelle l'endpoint <code>/end</code> pour clore la session cote metier.
252-279	<code>deleteLiveRoom()</code> confirme l'action, coupe le broadcast, ferme la socket, supprime la room via REST puis renvoie vers <code>/live</code> .
281-292	<code>publishEditorUpdate()</code> envoie <code>live_update</code> sur la socket seulement si le texte n'est pas vide, si l'utilisateur est l'editeur et si la socket est prete.
294-305	<code>sendChatMessage()</code> envoie <code>chat_message</code> a la room. Le front n'ecrit pas localement le message tant que le serveur ne l'a pas rebroadcast.

5.11 Chargement de room et orchestration des evenements socket

Fichier: `frontend/src/app/features/live/live-room-page/live-room-page.ts` | Lignes: 307-463

Ce bloc est le cerveau du composant. Il charge la room, ouvre la WebSocket au bon moment, puis route chaque message vers l'action a entreprendre.

```

307 private loadLiveRoom(eventId: number): void {
308   this.loading = true;
309   this.error = '';
310
311   this.liveRoomService
312     .getLiveRoomById(eventId)
313     .pipe(finalize(() => (this.loading = false)))
314     .subscribe({
315       next: (event) => {
316         this.event = event;
317         this.updates = [...event.updates].reverse();
318         this.chatMessages = [...event.chatMessages];
319         this.streamMessage =
320           event.status === 'live'
321             ? 'The room is live, wait for the editor video to connect.'
322             : event.status === 'ended'
323               ? 'This live session has ended.'
324               : 'The editor has not started the live session yet.';
325         this.attemptSocketConnection();
326       },
327       error: () => {
328         this.error = 'unable to load this live room right now.';
329       }
330     });
331 }
332
333 private attemptSocketConnection(): void {
334   if (this.event || this.currentUser || this.canJoinLiveRoom || this.isSocketReady || this.isSocketConnecting) {
335     return;
336   }
337
338   const token = this.authService.getToken();
339   if (!token) {
340     return;
341   }
342
343   this.connectionLabel = 'connecting...';
344   this.isSocketConnecting = true;
345   this.liveRoomSocketService.connect(this.event.id, token);
346 }
347
348 private disconnectSocket(): void {
349   this.isSocketReady = false;
350   this.isSocketConnecting = false;
351   this.socketClientId = '';
352   this.connectionLabel = 'offline';
353   this.liveRoomSocketService.disconnect();
354 }
355
356 private async handleSocketEvent(event: LiveSocketEvent): Promise<void> {
357   if (!this.event) {
358     return;
359   }
360
361   if (event.type === 'socket_ready') {
362     this.socketClientId = event.clientId;
363     this.isSocketReady = true;
364     this.isSocketConnecting = false;
365     this.connectionLabel = 'connected';
366     this.event = {
367       ...this.event,
368       viewer_count: event.viewerCount,
369       status: event.status
370     };
371
372     if (this.liveRoomEditor && this.event.status === 'live') {
373       this.requestViewerConnection();
374     }
375     return;
376   }
377
378   if (event.type === 'viewer_count') {
379     this.event = {
380       ...this.event,
381       viewer_count: event.viewerCount
382     };
383     return;
384   }
385
386   if (event.type === 'room_status') {
387     this.event = {
388       ...this.event,
389       status: event.status
390     };
391
392     if (event.status === 'live') {
393       this.streamMessage = this.liveRoomEditor
394         ? 'The room is live. Start the camera broadcast whenever you are ready.'
395         : 'The room is live, waiting for the editor camera stream.';
396       this.requestViewerConnection();
397     } else if (event.status === 'ended') {
398       this.streamMessage = 'This live session has ended.';
399       this.clearRoomViewerState();
400     }
401     return;
402   }
403
404   if (event.type === 'chat_message') {
405     this.chatMessages = [...this.chatMessages, event.message];
406     return;
407   }
408
409   if (event.type === 'live_update') {
410     this.updates = [event.message, ...this.updates];
411     return;
412   }
413
414   if (event.type === 'broadcast_ready') {
415     if (this.liveRoomEditor && this.event.status === 'live') {
416       this.broadcastSocketConnection = false;
417       this.requestViewerConnection();
418     }
419     return;
420   }
421
422   if (event.type === 'viewer_joined' && this.liveRoomEditor && this.isBroadcasting) {
423     await this.createPeerForViewer(event.senderClientId);
424     return;
425   }
426
427   if (event.type === 'offer' && this.liveRoomEditor) {
428     await this.handleAnswerFor(event.senderClientId, event.offer);
429     return;
430   }
431
432   if (event.type === 'answer' && this.liveRoomEditor) {
433     const peerConnection = this.broadcastPeerConnections.get(event.senderClientId);
434     if (!peerConnection) {
435       return;
436     }
437     await peerConnection.setRemoteDescription(new RTCSessionDescription(event.offer));
438     return;
439   }
440
441   if (event.type === 'ice_candidate') {
442     if (this.liveRoomEditor) {
443       const peerConnection = this.broadcastPeerConnections.get(event.senderClientId);
444       if (!peerConnection) {
445         return;
446       }
447       await peerConnection.addIceCandidate(new RTCIceCandidate(event.candidate));
448       return;
449     }
450
451     if (this.viewerPeerConnection) {
452       await this.viewerPeerConnection.addIceCandidate(new RTCIceCandidate(event.candidate));
453     }
454     return;
455   }
456
457   if (event.type === 'stream_stopped') {
458     this.streamMessage = 'The editor has stopped the camera stream.';
459     this.clearRoomViewerState();
460   }
461 }

```

Capture de code: 5.11 Chargement de room et orchestration des evenements socket

Lignes	Explication
307-331	loadLiveRoom(eventId) recupere le detail depuis le backend, alimente event, updates, chatMessages et fixe le texte d'etat selon upcoming/live/ended.
325	Une fois la room chargee, attemptSocketConnection() est appele. La connexion temps reel depend donc d'abord du chargement metier.
333-346	attemptSocketConnection() coupe toute tentative si la room n'est pas chargee, si l'utilisateur n'est pas logge, si l'accès n'est pas autorise, ou si une connexion existe deja.

Lignes	Explication
338-340	Le JWT est recupere via AuthService.getToken(). La WebSocket privée est donc strictement liée à une session valide.
343-345	Le label UI passe à Connecting..., puis le service ouvre /ws/live-events/{id}?token=...
348-354	disconnectSocket() reinitialise l'état local avant de fermer la socket réelle.
361-375	Branche socket_ready: la room recupere clientId, passe à l'état Connected, met à jour viewer_count et status, puis un viewer demande immédiatement une connexion si la room est déjà live.
378-384	Branche viewer_count: met simplement à jour le compteur affiche.
386-401	Branche room_status: si live, le message visuel change et le viewer essaie requestViewerConnection(); si ended, le flux distant est nettoye.
404-412	chat_message et live_update alimentent respectivement chatMessages et updates.
414-420	broadcaster_ready signifie que l'editeur diffuse sa camera. Le viewer remet hasRequestedViewerConnection à false puis relance requestViewerConnection() pour déclencher la negotiation.
422-429	viewer_joined et offer etablissent le basculement vers WebRTC: l'editeur cree une offre pour le viewer; le viewer la traite.
432-456	answer et ice_candidate terminent la negotiation progressive des pairs.
459-462	stream_ended remet l'interface viewer en mode attente et supprime remoteStream.

5.12 Signalisation WebRTC cote Angular

Fichier: frontend/src/app/features/live/live-room-page/live-room-page.ts | Lignes: 465-563

Ces fonctions transforment les messages WebSocket en vraie session WebRTC et branchent la video entre navigateur editeur et navigateur viewer.


```

Signalisation WebRTC cote Angular
Frontend/src/app/features/live/live-room-page/live-room-page.ts | lignes 465-563

465 private requestViewerConnection(): void {
466   if (this.isRoomEditor || !this.isSocketReady || this.hasRequestedViewerConnection) {
467     return;
468   }
469
470   this.hasRequestedViewerConnection = true;
471   this.liveRoomSocketService.send({
472     type: 'viewer_joined'
473   });
474 }
475
476 private async createOfferForViewer(targetClientId: string): Promise<void> {
477   if (!this.localStream) {
478     return;
479   }
480
481   const peerConnection = this.createBroadcasterPeerConnection(targetClientId);
482   const offer = await peerConnection.createOffer();
483   await peerConnection.setLocalDescription(offer);
484
485   this.liveRoomSocketService.send({
486     type: 'offer',
487     targetClientId,
488     sdp: offer
489   });
490 }
491
492 private createBroadcasterPeerConnection(targetClientId: string): RTCPeerConnection {
493   const existing = this.broadcasterPeerConnections.get(targetClientId);
494   if (existing) {
495     return existing;
496   }
497
498   const peerConnection = new RTCPeerConnection({
499     iceServers: [{ urls: 'stun:stun.l.google.com:19302' }]
500   });
501
502   this.localStream?.getTracks().forEach((track) => {
503     peerConnection.addTrack(track, this.localStream!);
504   });
505
506   peerConnection.onicecandidate = (event) => {
507     if (!event.candidate) {
508       return;
509     }
510
511     this.liveRoomSocketService.send({
512       type: 'ice_candidate',
513       targetClientId,
514       candidate: event.candidate.toJSON()
515     });
516   };
517
518   this.broadcasterPeerConnections.set(targetClientId, peerConnection);
519   return peerConnection;
520 }
521
522 private async handleViewerOffer(senderClientId: string, sdp: RTCSessionDescriptionInit): Promise<void> {
523   const peerConnection = this.createViewerPeerConnection(senderClientId);
524   await peerConnection.setRemoteDescription(new RTCSessionDescription(sdp));
525
526   const answer = await peerConnection.createAnswer();
527   await peerConnection.setLocalDescription(answer);
528
529   this.liveRoomSocketService.send({
530     type: 'answer',
531     targetClientId: senderClientId,
532     sdp: answer
533   });
534 }
535
536 private createViewerPeerConnection(targetClientId: string): RTCPeerConnection {
537   if (this.viewerPeerConnection) {
538     return this.viewerPeerConnection;
539   }
540
541   this.viewerPeerConnection = new RTCPeerConnection({
542     iceServers: [{ urls: 'stun:stun.l.google.com:19302' }]
543   });
544
545   this.viewerPeerConnection.ontrack = (event) => {
546     this.remoteStream = event.streams[0] ?? null;
547     this.bindRemoteStream();
548     this.sendMessage('connected to the editor video stream');
549   };
550
551   this.viewerPeerConnection.onicecandidate = (event) => {
552     if (!event.candidate) {
553       return;
554     }
555
556     this.liveRoomSocketService.send({
557       type: 'ice_candidate',
558       targetClientId,
559       candidate: event.candidate.toJSON()
560     });
561   };
562
563   return this.viewerPeerConnection;

```

Capture de code: 5.12 Signalisation WebRTC cote Angular

Lignes	Explication
465-474	requestViewerConnection() ne s'exécute que pour un viewer, avec socket prête, et une seule fois à la fois. Elle envoie viewer_joined au serveur.
476-490	createOfferForViewer(targetClientId) crée une offre SDP à partir de la RTCPeerConnection de l'éditeur et l'envoie au viewer cible.
492-500	createBroadcasterPeerConnection() réutilise une connexion existante si elle existe déjà; sinon il crée une nouvelle RTCPeerConnection avec un serveur STUN public.

Lignes	Explication
502-504	Toutes les pistes du localStream sont ajoutees a la connexion. C'est ici que la camera et le micro de l'editeur deviennent partageables.
506-516	onicecandidate capture chaque candidat ICE et le renvoie via WebSocket au viewer cible.
518-519	La connexion editeur est conservee dans broadcasterPeerConnections, une map par viewer.
522-534	handleViewerOffer(senderClientId, sdp) cree ou recupere la connexion viewer, applique l'offre distante, cree une answer puis la renvoie.
536-543	createViewerPeerConnection() garantit une unique connexion cote viewer.
545-549	ontrack est le point exact d'arrivee du flux. remoteStream recoit event.streams[0], bindRemoteStream() affecte la balise video et le texte d'etat passe a Connected to the editor video stream.
551-560	Le viewer renvoie aussi ses candidats ICE via WebSocket.

5.13 Nettoyage des flux et liaison aux balises video

Fichier: frontend/src/app/features/live/live-room-page/live-room-page.ts | Lignes: 566-619

Ce dernier bloc ferme proprement les flux et raccorde les MediaStream aux elements HTMLVideoElement.

Lignes	Explication
566-585	stopCameraBroadcast() annonce stream_ended s'il existait une diffusion, remet les drapeaux de camera a false, ferme les peer connections de l'editeur, stoppe les tracks locales et nettoie la video locale.
587-590	closeBroadcasterPeerConnections() ferme toutes les connexions editeur -> viewer puis vide la map.
592-600	clearRemoteViewerState() remet le viewer dans un etat sans flux: la connexion est closee, remoteStream est null et la balise video est videe.
603-610	bindLocalStream() attache localStream a la video locale puis essaye de lancer play().
612-619	bindRemoteStream() fait la meme chose pour remoteStream cote viewer.

6. Lecture detaillee du backend

6.1 Modeles SQLAlchemy du live

Fichier: backend/models.py | Lignes: 96-127

Le backend stocke la room et les messages dans deux tables simples. Toute la negotiation WebRTC reste hors base et vit uniquement en memoire navigateur/serveur.

```
Modeles SQLAlchemy LiveEvent et LiveMessage
backend/models.py | lignes 96-127

96 class LiveEvent(Base):
97     __tablename__ = "live_events"
98
99     id = Column(Integer, primary_key=True, index=True)
100     title = Column(String(200), nullable=False)
101     description = Column(Text, nullable=False)
102     category = Column(String(50), nullable=False)
103     cover_image = Column(String(500), nullable=True)
104     stream_url = Column(String(500), nullable=True)
105     status = Column(String(20), nullable=False, default="upcoming")
106     premium_only = Column(Boolean, nullable=False, default=True)
107     editor_user_id = Column(Integer, ForeignKey("users.id", ondelete="SET NULL"), nullable=True)
108     created_at = Column(DateTime, server_default=func.now())
109     started_at = Column(DateTime, nullable=True)
110     ended_at = Column(DateTime, nullable=True)
111
112     editor = relationship("User", back_populates="live_events")
113     messages = relationship("LiveMessage", back_populates="live_event", cascade="all, delete-orphan")
114
115
116 class LiveMessage(Base):
117     __tablename__ = "live_messages"
118
119     id = Column(Integer, primary_key=True, index=True)
120     live_event_id = Column(Integer, ForeignKey("live_events.id", ondelete="CASCADE"), nullable=False)
121     user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
122     message_type = Column(String(20), nullable=False)
123     content = Column(Text, nullable=False)
124     created_at = Column(DateTime, server_default=func.now())
125
126     live_event = relationship("LiveEvent", back_populates="messages")
127     user = relationship("User", back_populates="live_messages")
```

Capture de code: 6.1 Modeles SQLAlchemy du live

Lignes	Explication
96-110	LiveEvent contient les metadonnees de la room: titre, description, categorie, image, stream_url, status, premium_only, l'editeur propriétaire et les timestamps de vie.
112-113	Les relations editor et messages relient la room au User propriétaire et a l'historique des messages.
116-124	LiveMessage stocke chaque message avec son live_event_id, son user_id, son type et son contenu.
126-127	Les relations renvoient vers la room et vers l'utilisateur auteur.

6.2 Schemas Pydantic utiles au live

Fichier: backend/schemas.py | Lignes: 114-124

Les schemas de creation sont volontairement legers: la vraie complexite du temps reel ne passe pas par Pydantic mais par la WebSocket.

Lignes	Explication
114-120	CreateLiveEventRequest exige titre, description, categorie, accepte optionnellement cover_image et stream_url, puis premium_only.
123-124	CreateLiveMessageRequest ne contient qu'un content. Dans la pratique, le projet utilise surtout des payloads JSON recus dans la WebSocket.

6.3 Authentification et garde d'accès côté serveur

Fichier: backend/main.py | Lignes: 141-179

Avant de laisser entrer un client dans la room, le serveur valide le JWT, vérifie le rôle editor si nécessaire, puis contrôle l'accès premium.

Lignes	Explication
141-152	get_current_user_from_token(token, db) decode le JWT, retrouve ou crée l'utilisateur et renvoie un objet User exploitable par la WebSocket.
160-162	ensure_editor(current_user) garantit que certaines actions sensibles restent réservées au rôle editor.
165-168	ensure_live_room_editor(live_event, current_user) ajoute une vérification de propriété: la room doit appartenir à cet éditeur.
171-179	can_access_live_event(live_event, current_user) autorise toujours l'éditeur propriétaire, puis applique la règle premium_only pour les autres.

6.4 Serialisation des messages live

Fichier: backend/main.py | Lignes: 232-240

Cette petite fonction convertit un LiveMessage SQLAlchemy en dictionnaire JSON exploitable par le frontend.

Lignes	Explication
232-240	serialize_live_message() expose id, message_type, contenu, date ISO, user_id et user_name. Ce format est ensuite réemployé dans chat_message et live_update.

6.5 LiveRoomManager: connexions WebSocket en mémoire

Fichier: backend/main.py | Lignes: 243-302

Cette capture correspond exactement au gestionnaire en mémoire des connexions WebSocket par room.

```
Gestionnaire serveur des connexions WebSocket
backend/main.py | lignes 243-302

243 class LiveRoomManager:
244     def __init__(self) -> None:
245         self.connections: dict[int, dict[str, dict]] = defaultdict(dict)
246
247     async def connect(self, room_id: int, websocket: WebSocket, user: models.User) -> str:
248         client_id = str(uuid4())
249         self.connections[room_id][client_id] = {
250             "socket": websocket,
251             "user_id": user.id,
252             "role": user.role or "user",
253         }
254         return client_id
255
256     def disconnect(self, room_id: int, client_id: str) -> None:
257         room_connections = self.connections.get(room_id)
258         if room_connections is None:
259             return
260
261         room_connections.pop(client_id, None)
262         if not room_connections:
263             self.connections.pop(room_id, None)
264
265     def get_viewer_count(self, room_id: int) -> int:
266         return len(self.connections.get(room_id, {}))
267
268     async def broadcast(self, room_id: int, payload: dict, exclude_client_id: Optional[str] = None) -> None:
269         room_connections = self.connections.get(room_id, {})
270         disconnected_clients: list[str] = []
271
272         for client_id, connection in room_connections.items():
273             if exclude_client_id and client_id == exclude_client_id:
274                 continue
275
276             try:
277                 await connection["socket"].send_json(payload)
278             except Exception:
279                 disconnected_clients.append(client_id)
280
281         for client_id in disconnected_clients:
282             self.disconnect(room_id, client_id)
283
284     async def send_to_client(self, room_id: int, client_id: str, payload: dict) -> None:
285         room_connections = self.connections.get(room_id, {})
286         target = room_connections.get(client_id)
287         if not target:
288             return
289
290         try:
291             await target["socket"].send_json(payload)
292         except Exception:
293             self.disconnect(room_id, client_id)
294
295     async def broadcast_viewer_count(self, room_id: int) -> None:
296         await self.broadcast(
297             room_id,
298             {
299                 "type": "viewer_count",
300                 "viewerCount": self.get_viewer_count(room_id),
301             },
302         )
```

Capture de code: 6.5 LiveRoomManager: connexions WebSocket en memoire

Lignes	Explication
243-245	Le constructeur initialise connections comme un dictionnaire de rooms, chacune contenant un sous-dictionnaire clientId -> meta connexion.
247-254	connect(room_id, websocket, user) cree un clientId UUID et l'enregistre avec le socket, l'user_id et le role.
256-263	disconnect(room_id, client_id) retire une connexion. Si la room n'a plus aucun client, elle est supprimee du manager.
265-266	get_viewer_count(room_id) compte les connexions actives pour cette room.

Lignes	Explication
268-282	<code>broadcast(room_id, payload, exclude_client_id)</code> envoie un JSON a tous les clients de la room, avec possibilite d'exclure l'emetteur. Les sockets cassees sont nettoyees ensuite.
284-293	<code>send_to_client(room_id, client_id, payload)</code> permet le routage cible vers un pair precis, indispensable pour <code>offer</code> , <code>answer</code> et <code>ice_candidate</code> .
295-302	<code>broadcast_viewer_count(room_id)</code> diffuse le compteur a tous les clients de la room.

6.6 Serialization des rooms et de leur detail

Fichier: `backend/main.py` | Lignes: 308-343

Ces fonctions transforment une room en JSON simple ou detaille, avec injection du compteur de connexions actives.

Lignes	Explication
308-324	<code>serialize_live_event()</code> expose la room avec <code>viewer_count</code> injecte depuis la memoire et les timestamps ISO.
327-343	<code>serialize_live_event_detail()</code> separe l'historique update de l'historique chat et ne renvoie que les 30 updates et 40 messages les plus recents.

6.7 Endpoints REST du module live

Fichier: `backend/main.py` | Lignes: 410-579

La couche REST pilote le cycle de vie metier de la room. Elle ne transporte pas le flux camera, mais fixe le contexte a partir duquel le temps reel fonctionne.

```

410 @app.get("/live-events")
411 def list_live_events(db: Session = Depends(database.get_db)):
412     events = []
413     db_query(models.LiveEvent)
414     order_by
415     desc(models.LiveEvent.status == "live"),
416     desc(models.LiveEvent.created_at),
417     }
418     .all()
419     }
420     }
421     return [
422         serialize_live_event(event, viewer_count_live_room_manager.get_viewer_count(event_id))
423         for event in events
424     ]
425
426 @app.get("/live-events/{event_id}")
427 def get_live_event(event_id: int, db: Session = Depends(database.get_db)):
428     event = db.query(models.LiveEvent).filter(models.LiveEvent.id == event_id).first()
429     if not event:
430         raise HTTPException(status_code=404, detail="Live event not found.")
431     return serialize_live_event_detail(event)
432
433 @app.post("/live-events")
434 def create_live_event(
435     payload: schemas.CreateLiveEventRequest,
436     db: Session = Depends(database.get_db),
437     current_user: models.User = Depends(get_current_user),
438 ):
439     try:
440         ensure_editor(current_user)
441
442         title = payload.title.strip()
443         description = payload.description.strip()
444         category = payload.category.strip().lower()
445
446         if not title:
447             raise HTTPException(status_code=400, detail="A live room title is required.")
448         if not description:
449             raise HTTPException(status_code=400, detail="A live room description is required.")
450         if not category:
451             raise HTTPException(status_code=400, detail="A category is required.")
452
453         live_event = models.LiveEvent(
454             title=title,
455             description=description,
456             category=category,
457             viewer_count_live_room_manager.get_viewer_count(payload.stream_url) if payload.stream_url else None,
458             stream_url=payload.stream_url.strip() if payload.stream_url else None,
459             previous_only=payload.previous_only,
460             editor_user_id=current_user.id,
461             status="upcoming",
462         )
463         db.add(live_event)
464         db.commit()
465         db.refresh(live_event)
466         return serialize_live_event(event)
467     except HTTPException as he:
468         db.rollback()
469         raise he
470     except Exception as exc:
471         db.rollback()
472         raise HTTPException(status_code=500, detail="Live room creation failed. (exc)")
473
474 @app.post("/live-events/{event_id}/start")
475 def start_live_event(
476     event_id: int,
477     db: Session = Depends(database.get_db),
478     current_user: models.User = Depends(get_current_user),
479 ):
480     try:
481         event = db.query(models.LiveEvent).filter(models.LiveEvent.id == event_id).first()
482         if not event:
483             raise HTTPException(status_code=404, detail="Live event not found.")
484         ensure_live_room_editor(event, current_user)
485
486         event.status = "live"
487         event.start_at = datetime.utcnow()
488         event.end_at = None
489
490         db.commit()
491         db.refresh(event)
492
493         # Notify live room manager
494         event_id =
495         {
496             "type": "room status",
497             "status": event.status,
498         },
499     ]
500     return serialize_live_event(event, viewer_count_live_room_manager.get_viewer_count(event_id))
501     except HTTPException as he:
502         db.rollback()
503         raise he
504     except Exception as exc:
505         db.rollback()
506         raise HTTPException(status_code=500, detail="Unable to start the live room. (exc)")
507
508 @app.post("/live-events/{event_id}/end")
509 def end_live_event(
510     event_id: int,
511     db: Session = Depends(database.get_db),
512     current_user: models.User = Depends(get_current_user),
513 ):
514     try:
515         event = db.query(models.LiveEvent).filter(models.LiveEvent.id == event_id).first()
516         if not event:
517             raise HTTPException(status_code=404, detail="Live event not found.")
518         ensure_live_room_editor(event, current_user)
519
520         event.status = "ended"
521         event.end_at = datetime.utcnow()
522
523         db.commit()
524         db.refresh(event)
525
526         # Notify live room manager
527         event_id =
528         {
529             "type": "room status",
530             "status": event.status,
531         },
532     ]
533     # Notify live room manager
534     {
535         "type": "stream ended",
536     },
537     ]
538     return serialize_live_event(event, viewer_count_live_room_manager.get_viewer_count(event_id))
539     except HTTPException as he:
540         db.rollback()
541         raise he
542     except Exception as exc:
543         db.rollback()
544         raise HTTPException(status_code=500, detail="Unable to end the live room. (exc)")
545
546 @app.delete("/live-events/{event_id}")
547 def delete_live_event(
548     event_id: int,
549     db: Session = Depends(database.get_db),
550     current_user: models.User = Depends(get_current_user),
551 ):
552     try:
553         event = db.query(models.LiveEvent).filter(models.LiveEvent.id == event_id).first()
554         if not event:
555             raise HTTPException(status_code=404, detail="Live event not found.")
556         ensure_live_room_editor(event, current_user)
557
558         db.delete(event)
559         db.commit()
560
561         live_room_manager.disconnect(event_id, None)
562
563         return {"message": "Live room deleted successfully."}
564     except HTTPException as he:
565         db.rollback()
566         raise he
567     except Exception as exc:
568         db.rollback()
569         raise HTTPException(status_code=500, detail="Unable to delete the live room. (exc)")

```

Capture de code: 6.7 Endpoints REST du module live

Lignes	Explication
410-424	list_live_events() recupere les rooms triees en mettant les rooms live en premier et injecte viewer_count a la volee.
427-433	get_live_event(event_id) charge le detail d'une room unique ou renvoie 404 si elle n'existe pas.
436-476	create_live_event() verifie que l'utilisateur est editor, nettoie les champs, cree la room avec status='upcoming' puis la persiste.

Lignes	Explication
479-512	start_live_event() verifie la room et l'editeur propriétaire, passe status='live', renseigne started_at, commit puis broadcast room_status=live a la room.
515-553	end_live_event() passe status='ended', renseigne ended_at, broadcast room_status=ended puis broadcast stream_ended.
556-579	delete_live_event() supprime la room en base et nettoie les connexions en memoire pour cette room.

6.8 Ouverture et validation du endpoint WebSocket

Fichier: backend/main.py | Lignes: 970-1008

Cette premiere capture du endpoint WebSocket couvre uniquement l'entree dans la room, les verifications et l'envoi de socket_ready.

```

Ouvverture et validation de la WebSocket serveur
backend/main.py | lignes 970-1008

970 @app.websocket("/ws/live-events/{event_id}")
971 async def live_event_socket(
972     websocket: WebSocket,
973     event_id: int,
974     token: str = Query(default=""),
975 ):
976     db = database.SessionLocal()
977     client_id: Optional[str] = None
978     room_id = event_id
979
980     try:
981         if not token:
982             await websocket.close(code=4401)
983             return
984
985         current_user = get_current_user_from_token(token, db)
986         live_event = db.query(models.LiveEvent).filter(models.LiveEvent.id == event_id).first()
987
988         if not live_event:
989             await websocket.close(code=4404)
990             return
991
992         if not can_access_live_event(live_event, current_user):
993             await websocket.close(code=4403)
994             return
995
996         await websocket.accept()
997         client_id = await live_room_manager.connect(room_id, websocket, current_user)
998
999         await websocket.send_json(
1000             {
1001                 "type": "socket_ready",
1002                 "clientId": client_id,
1003                 "viewerCount": live_room_manager.get_viewer_count(room_id),
1004                 "status": live_event.status,
1005                 "isEditor": live_event.editor_user_id == current_user.id and (current_user.role or "user") == "editor",
1006             }
1007         )
1008         await live_room_manager.broadcast_viewer_count(room_id)

```

Capture de code: 6.8 Ouverture et validation du endpoint WebSocket

Lignes	Explication
970-979	La fonction recoit websocket, event_id et token. Elle ouvre une session DB dediee a la duree de vie de la socket.

Lignes	Explication
980-994	Le serveur refuse toute connexion sans token, sans room existante ou sans droit d'accès. Les codes 4401, 4404 et 4403 explicitent la raison.
996-1008	Après <code>accept()</code> , le serveur enregistre la connexion dans <code>live_room_manager</code> , envoie <code>socket_ready</code> avec <code>clientId</code> , <code>viewerCount</code> , <code>status</code> et <code>isEditor</code> , puis broadcast le nouveau <code>viewer_count</code> .

6.9 WebSocket: chat, updates et annonces d'etat

Fichier: `backend/main.py` | Lignes: 1010-1085

Cette capture couvre les branches qui persistent ou diffusent les messages de room avant la partie purement WebRTC.

```
Branches chat, update et broadcaster_ready cote serveur
backend/main.py | lignes 1010-1085

1010     while True:
1011         payload = await websocket.receive_json()
1012         message_type = str(payload.get("type", "")).strip()
1013
1014         if message_type == "chat_message":
1015             content = str(payload.get("content", "")).strip()
1016             if not content:
1017                 continue
1018
1019             live_message = models.LiveMessage(
1020                 live_event_id=live_event.id,
1021                 user_id=current_user.id,
1022                 message_type="chat",
1023                 content=content,
1024             )
1025             db.add(live_message)
1026             db.commit()
1027             db.refresh(live_message)
1028             live_message.user = current_user
1029
1030             await live_room_manager.broadcast(
1031                 room_id,
1032                 {
1033                     "type": "chat_message",
1034                     "message": serialize_live_message(live_message),
1035                 },
1036             )
1037             continue
1038
1039         if message_type == "live_update":
1040             ensure_live_room_editor(live_event, current_user)
1041             content = str(payload.get("content", "")).strip()
1042             if not content:
1043                 continue
1044
1045             live_message = models.LiveMessage(
1046                 live_event_id=live_event.id,
1047                 user_id=current_user.id,
1048                 message_type="update",
1049                 content=content,
1050             )
1051             db.add(live_message)
1052             db.commit()
1053             db.refresh(live_message)
1054             live_message.user = current_user
1055
1056             await live_room_manager.broadcast(
1057                 room_id,
1058                 {
1059                     "type": "live_update",
1060                     "message": serialize_live_message(live_message),
1061                 },
1062             )
1063             continue
1064
1065         if message_type == "broadcaster_ready":
1066             ensure_live_room_editor(live_event, current_user)
1067             await live_room_manager.broadcast(
1068                 room_id,
1069                 {
1070                     "type": "broadcaster_ready",
1071                     "senderClientId": client_id,
1072                 },
1073                 exclude_client_id=client_id,
1074             )
1075             continue
1076
1077         if message_type == "stream_ended":
1078             ensure_live_room_editor(live_event, current_user)
1079             await live_room_manager.broadcast(
1080                 room_id,
1081                 {
1082                     "type": "stream_ended",
1083                 },
1084             )
1085             continue
```

Capture de code: 6.9 WebSocket: chat, updates et annonces d'etat

Lignes	Explication
1010-1013	La boucle attend un payload JSON et lit message_type pour savoir quelle branche executer.
1014-1037	Branche chat_message: le message est nettoye, persiste en base comme type='chat', puis rebroadcast a toute la room.
1039-1063	Branche live_update: reservee a l'editeur propriétaire, persiste le message comme type='update', puis diffuse l'update officielle.

Lignes	Explication
1065-1075	Branche broadcaster_ready: reservee a l'editeur. Le serveur diffuse l'information a tous les autres clients de la room, sans se la renvoyer.
1077-1085	Branche stream_ended: reservee a l'editeur. Le serveur diffuse la fin du flux camera.

6.10 WebSocket: signalisation WebRTC et nettoyage final

Fichier: backend/main.py | Lignes: 1087-1114

Cette derniere capture montre le relai des messages WebRTC et le nettoyage de la connexion quand un client se deconnecte.

●●● Relai du signaling WebRTC et nettoyage serveur

backend/main.py | lignes 1087-1114

```
1087         if message_type in {"viewer_joined", "offer", "answer", "ice_candidate"}:
1088             outgoing_payload = {
1089                 "type": message_type,
1090                 "senderClientId": client_id,
1091             }
1092
1093             if message_type in {"offer", "answer"}:
1094                 outgoing_payload["sdp"] = payload.get("sdp")
1095
1096             if message_type == "ice_candidate":
1097                 outgoing_payload["candidate"] = payload.get("candidate")
1098
1099             target_client_id = payload.get("targetClientId")
1100             if target_client_id:
1101                 await live_room_manager.send_to_client(room_id, str(target_client_id), outgoing_payload)
1102             else:
1103                 await live_room_manager.broadcast(
1104                     room_id,
1105                     outgoing_payload,
1106                     exclude_client_id=client_id,
1107                 )
1108         except WebSocketDisconnect:
1109             pass
1110         finally:
1111             if client_id is not None:
1112                 live_room_manager.disconnect(room_id, client_id)
1113                 await live_room_manager.broadcast_viewer_count(room_id)
1114             db.close()
```

Capture de code: 6.10 WebSocket: signalisation WebRTC et nettoyage final

Lignes	Explication
1087-1098	Le serveur prepare un payload sortant commun pour viewer_joined, offer, answer et ice_candidate, puis ajoute sdp ou candidate selon le cas.
1099-1107	Si targetClientId est present, le message est route vers un pair precis. Sinon, il est diffuse aux autres clients de la room.
1108-1114	Au disconnect, la socket est retiree du manager, le viewer_count est rediffuse, puis la session DB est fermee.

7. Resume final du fonctionnement du live

Le module live de NewsHub suit une logique tres nette. Le hub /live sert a creer ou choisir une room. Quand la room /live/:id s'ouvre, Angular charge son etat initial via REST. Si l'utilisateur a le droit d'entrer, une WebSocket privée est ensuite ouverte avec un JWT. La WebSocket apporte l'etat courant de la room, le chat, les updates, le compteur de lecteurs connectes et la signalisation.

Le media n'est pas transporte par la WebSocket. Quand l'editeur clique sur Go live with camera, le front marque la diffusion comme active et envoie broadcaster_ready. Les lecteurs envoient alors viewer_joined, puis l'offre, la reponse et les candidats ICE circulent via la socket. Enfin, l'evenement ontrack du lecteur raccorde le vrai flux distant a la balise video. C'est exactement la que le lecteur rejoint le direct video.

En bref: REST installe la scene, WebSocket dirige la session, WebRTC transporte l'image et le son. L'implementation est lisible, pedagogique et bien separee entre controle metier et flux media.